

Algorithms

Interview Cracker question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

Complexity Analysis

If you get one subject right, make it this one. Every other answer you give will be followed by "and what's the complexity?"

[Easy] Q. What does Big-O actually describe?

Answer. An upper bound on how the cost of an algorithm grows as the input grows, ignoring constant factors and lower-order terms.

It deliberately throws away detail. $O(n)$ says "doubling the input roughly doubles the work". It does not say the algorithm is fast - an $O(n)$ algorithm with a huge constant can lose to an $O(n \log n)$ one at every realistic size.

The three you will be asked to distinguish:

- O - upper bound ("no worse than").
- Ω - lower bound ("no better than").
- Θ - both, a tight bound.

In practice interviewers say "Big-O" but mean Θ . Answer with the tight bound and you will never be wrong.

[Easy] Q. Order these from fastest-growing to slowest: $O(n \log n)$, $O(1)$, $O(2^n)$, $O(n^2)$, $O(\log n)$, $O(n)$

Answer. From slowest-growing (best) to fastest-growing (worst):

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

A rough feel for what each survives, at roughly a second of compute:

Complexity	Largest practical n
$O(\log n)$	effectively unbounded
$O(n)$	~100,000,000
$O(n \log n)$	~5,000,000
$O(n^2)$	~10,000
$O(2^n)$	~25
$O(n!)$	~11

This table is the real reason constraints appear in problem statements. If the input is 10^5 , an $O(n^2)$ solution will not pass and you should stop and look for an $O(n \log n)$ one.

[Medium] Q. What is amortised complexity, and why is appending to a dynamic array $O(1)$?

Answer. Amortised complexity is the average cost per operation across a worst-case sequence of operations - not the average over random inputs.

A dynamic array (Python `list`, Java `ArrayList`, Go `slice`, C++ `vector`) keeps spare capacity. Appending is normally a write to a free slot: $O(1)$. When capacity runs out it allocates a bigger block - usually double - and copies everything, which is $O(n)$.

The doubling is what makes it work. To reach n elements the copies cost $1 + 2 + 4 + \dots + n$, which sums to less than $2n$. Spread across n appends that is $O(1)$ each.

The follow-up: what if it grew by a constant 10 instead of doubling? Then reaching n costs $10 + 20 + 30 + \dots$, an arithmetic series that sums to $O(n^2)$, so each append is $O(n)$ amortised. Geometric growth is the whole trick.

[Medium] Q. What is the difference between amortised, average-case and worst-case?

Answer. They answer different questions:

- **Worst case** - the most expensive single input. Hash table lookup: $O(n)$,
- **Average case** - expected cost over a distribution of inputs. Hash table
- **Amortised** - guaranteed average across a sequence, with no probability

The distinction that catches people out: amortised is a guarantee, average is a statistical expectation. A quicksort's $O(n \log n)$ is average-case - an adversary can still force $O(n^2)$. A dynamic array's $O(1)$ append is amortised - no adversary can break it.

[Easy] Q. How do you analyse nested loops?

Answer. Multiply the iteration counts, but read the bounds carefully.

```
for i in range(n):           # n
    for j in range(n):       # n
        work()               # O(1)
# O(n^2)
```

The one that trips people up:

```
for i in range(n):
    for j in range(i, n):    # shrinking inner loop
        work()
```

That is $n + (n-1) + \dots + 1 = n(n+1)/2$, which is still $O(n^2)$. Halving the work does not change the growth class.

And the one that looks quadratic but is not:

```
for i in range(n):
    j = 1
    while j < n:
        j *= 2                # log n iterations, not n
```

That is $O(n \log n)$.

[Medium] Q. What is the complexity of this, and why?

```
def f(s):
    result = ""
    for ch in s:
        result += ch
    return result
```

Answer. $O(n^2)$ in languages with immutable strings (Python, Java, C#, JavaScript).

`result += ch` cannot extend the existing string - strings are immutable, so it allocates a new string and copies every character across. Doing that n times copies $1 + 2 + \dots + n$ characters, which is $O(n^2)$.

The fix is to collect the pieces and join once, which is $O(n)$:

```
return "".join(s)          # Python
# StringBuilder in Java/C#, array.push + join in JavaScript
```

This is a genuinely common interview trap, because the $O(n^2)$ version looks like a simple linear loop.

[Medium] Q. How do you reason about the complexity of a recursive function?

Answer. Count the nodes in the recursion tree, then multiply by the work done per node.

Fibonacci, naive:

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Each call spawns two, to a depth of n , so roughly 2^n nodes with $O(1)$ work each: **$O(2^n)$ time, $O(n)$ space** (the deepest path of the call stack).

Merge sort: the tree has $\log n$ levels, and every level touches all n elements during the merges: **$O(n \log n)$** .

Binary search: one child per call, $\log n$ deep, $O(1)$ per node: **$O(\log n)$** .

For divide-and-conquer, the Master Theorem formalises this, but the recursion tree gets you there and is easier to explain out loud.

[Medium] Q. What is space complexity, and does the call stack count?

Answer. Space complexity is the extra memory an algorithm needs beyond its input - and yes, the call stack counts.

A recursive function that is $O(n)$ deep uses $O(n)$ space even if it allocates nothing, which is exactly why deep recursion causes stack overflow. Converting to iteration with an explicit stack does not reduce the space, but it does move it from the (small, fixed) stack to the (large) heap.

Common values worth memorising:

Algorithm	Time	Space
Merge sort	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$ avg	$O(\log n)$ stack
Heap sort	$O(n \log n)$	$O(1)$
BFS / DFS on a graph	$O(V + E)$	$O(V)$
Binary search, iterative	$O(\log n)$	$O(1)$

Heap sort being the only $O(1)$ -space $O(n \log n)$ comparison sort is a favourite follow-up.

Dynamic Programming

The topic candidates fear most. It is far more learnable than its reputation - almost every DP problem is the same four steps applied to a different state.

[Easy] Q. What is dynamic programming?

Answer. Solving a problem by breaking it into overlapping subproblems and storing each subproblem's answer so it is computed only once.

Two properties must hold, and naming both is the mark of a real answer:

- 1 **Optimal substructure** - the optimal solution is built from optimal
- 2 **Overlapping subproblems** - the same subproblem recurs many times.

If subproblems don't overlap, you have divide and conquer (merge sort), not DP. If there is no optimal substructure, DP does not apply at all.

[Medium] Q. Memoisation vs tabulation?

Answer. Two ways to implement the same recurrence.

Memoisation (top-down) - ordinary recursion plus a cache.

```
from functools import cache

@cache
def fib(n):
    return n if n <= 1 else fib(n - 1) + fib(n - 2)
```

Tabulation (bottom-up) - fill a table from the base cases up.

```
def fib(n):
    if n <= 1:
        return n
    prev, curr = 0, 1
    for _ in range(n - 1):
        prev, curr = curr, prev + curr
    return curr
```

	Memoisation	Tabulation
Written as	Recursion + cache	Loops
Computes	Only reachable states	Every state
Space	Table + O(depth) call stack	Table, often reducible
Risk	Stack overflow when deep	None
Easier to	Write from the recurrence	Optimise the space of

Write the memoised version first - it follows directly from the recurrence and is much harder to get wrong. Convert to tabulation if you need the space optimisation or are worried about stack depth.

[Medium] Q. How do you recognise a DP problem?

Answer. Signals, in rough order of reliability:

- It asks for a **count** of ways, or a **minimum/maximum**, or whether something
- The brute force is **exponential** and recomputes the same states.
- It involves a **subsequence** (not subarray), or choices of take/skip.
- There are small numeric constraints that look like array dimensions ($n \leq 1000$)

Counter-signal: if a locally optimal choice is provably globally optimal, it is a **greedy** problem, which is simpler. Interval scheduling by earliest finish time is greedy; the knapsack is not.

[Hard] Q. Walk through solving a DP problem from scratch.

Answer. Four steps. Do them in this order, out loud.

Problem: climbing stairs, taking 1 or 2 steps at a time - how many distinct ways to reach step n ?

1. Define the state. $\text{ways}(i)$ = the number of distinct ways to reach step i . Getting this sentence precise is 80% of the work.

2. Find the recurrence. You arrive at step i either from $i-1$ (a 1-step) or from $i-2$ (a 2-step), and those routes are disjoint:

```
ways(i) = ways(i - 1) + ways(i - 2)
```

3. Base cases. $\text{ways}(0) = 1$ (one way to stand still), $\text{ways}(1) = 1$.

4. Order of evaluation. i depends on smaller i , so iterate upward.

```
def climb(n):
    prev, curr = 1, 1                # ways(0), ways(1)
    for _ in range(n - 1):
        prev, curr = curr, prev + curr
    return curr
```

$O(n)$ time, $O(1)$ space - because each state needs only the previous two, the table collapses to two variables. That collapse is the most common DP optimisation and interviewers look for it.

[Hard] Q. Explain the 0/1 knapsack.

Answer. Given items with weights and values and a capacity, maximise value without exceeding capacity. Each item is taken at most once - the "0/1".

State: $\text{best}[i][c]$ = the best value using the first i items with capacity c .

Recurrence - for each item, take it or skip it:

```
best[i][c] = max(
    best[i-1][c],                # skip
    best[i-1][c - weight[i]] + value[i]  # take, if it fits
)

def knapsack(weights, values, capacity):
    best = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):  # BACKWARDS
            best[c] = max(best[c], best[c - w] + v)
    return best[capacity]
```

$O(n \times \text{capacity})$ time, $O(\text{capacity})$ space.

The backwards inner loop is the subtle part. Iterating forwards would let the same item be used twice, because `best[c - w]` would already have been updated in this round. Forwards gives you the **unbounded** knapsack (unlimited copies); backwards gives 0/1. Being able to explain that one line is a strong signal.

Is it polynomial? No - this is pseudo-polynomial. `capacity` is a numeric value, not an input size, so the table is exponential in the number of bits used to write the capacity. 0/1 knapsack is NP-hard.

[Medium] Q. What are the standard DP patterns worth recognising?

Answer. Most problems are a variation of one of these.

Pattern	State	Examples
Linear	<code>dp[i]</code> from <code>dp[i-1]</code> , <code>dp[i-2]</code>	Climbing stairs, house robber, max subarray
Two sequences	<code>dp[i][j]</code> over both	Edit distance, longest common subsequence
Knapsack	<code>dp[item][capacity]</code>	Subset sum, coin change, partition
Interval	<code>dp[left][right]</code>	Matrix chain, burst balloons, palindromes
Grid	<code>dp[row][col]</code>	Unique paths, minimum path sum
State machine	<code>dp[i][state]</code>	Stock trading with cooldown or fees

Naming the pattern early ("this is a two-sequence DP, so I want a 2-D table indexed by positions in each string") makes the rest follow mechanically.

[Medium] Q. What is the difference between DP and greedy?

Answer. Greedy commits to the locally best choice and never reconsiders. DP explores all choices and keeps the best.

Greedy is faster (usually $O(n \log n)$ vs $O(n^2)$ or worse) but is only correct when the problem has the **greedy-choice property** - that a locally optimal choice is part of some globally optimal solution.

The classic demonstration is coin change. Greedy (take the largest coin that fits) works for `[1, 5, 10, 25]`: for 30 it gives `25 + 5`, which is optimal. It fails for `[1, 3, 4]`: for 6 it gives `4 + 1 + 1 = three` coins, but `3 + 3 = two` is optimal. Same algorithm, different coin set, wrong answer.

So: if you propose greedy, you owe a justification or a proof sketch. If you cannot give one, use DP. Never assert greedy works because it passes your two hand-picked examples.

Number Theory & Math

The math questions that actually come up: GCD, modular arithmetic, primality, and a handful of bit tricks. None of them require advanced theory - they require knowing which tool fits which shape of problem.

[Easy] Q. How does the Euclidean algorithm compute a GCD, and why is it fast?

Answer. $\text{gcd}(a, b) == \text{gcd}(b, a \% b)$, repeated until the remainder is 0. Each step shrinks the pair, and the remainder at least halves every two steps, so it runs in $O(\log(\min(a, b)))$ - which is why it is the workhorse behind fraction reduction, CRT, and RSA.

```
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

assert gcd(1071, 462) == 21
assert gcd(0, 5) == 5          # gcd(0, x) == x by convention
assert gcd(270, 192) == 6
```

The extended version also returns x, y with $a*x + b*y == \text{gcd}(a, b)$ - that identity is what makes modular inverses possible at all.

[Medium] Q. When do you need modular arithmetic, and how do you divide "mod p"?

Answer. Modulo keeps numbers bounded - hashing, cyclic buffers, "is it a power of two" checks, and big-prime arithmetic in cryptography. The trap is division: $a / b \bmod p$ is NOT $(a \bmod p) / (b \bmod p)$. You multiply by the **modular inverse** of b instead, which exists only when $\text{gcd}(b, p) == 1$ (always true when p is prime and b is not a multiple of it).

```
# "1000 * 7^(mod -1) mod 13" - i.e. 1000/7 mod 13, done by multiplying
# 1000 by the modular inverse of 7.
p = 13
assert pow(7, -1, p) == 2          # 7 * 2 = 14 = 1 (mod 13)
answer = (1000 % p) * pow(7, -1, p) % p
assert answer == 11                # 1000 * 2 = 2000 = 11 (mod 13)
```

When p is prime you can also get the inverse from Fermat's little theorem: $b^{(p-2)} \bmod p$. Python's `pow(base, exp, mod)` does fast modular exponentiation, so this is cheap.

[Medium] Q. Trial division vs a sieve - how do you test or generate primes?

Answer. To test ONE number, trial-divide up to its square root (skip evens after 2): $O(\sqrt{n})$. To generate ALL primes up to n , use the Sieve of Eratosthenes: mark multiples of each prime starting at its square, because smaller multiples were already crossed off. The sieve is roughly $O(n \log \log n)$ and beats testing each number individually by a wide margin once n is large.

```
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0:
        return False
    f = 3
    while f * f <= n:          # only need to test up to sqrt(n)
        if n % f == 0:
            return False
        f += 2
```



```

        f += 2
    return True

assert is_prime(2) and is_prime(3) and is_prime(7)
assert not is_prime(1) and not is_prime(0) and not is_prime(2**20)

def sieve(n: int) -> list[int]:
    prime = bytearray([1]) * (n + 1)
    prime[0:2] = b"\x00\x00"
    for i in range(2, int(n ** 0.5) + 1):
        if prime[i]:
            prime[i * i :: i] = bytearray(len(prime[i * i :: i]))
    return [i for i in range(n + 1) if prime[i]]

assert sieve(30) == [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

```

Starting each marking pass at $i*i$ (not $2*i$) is the detail interviewers probe for: everything below $i*i$ that is a multiple of i already has a smaller prime factor and was crossed off earlier.

[Medium] Q. What are the bit tricks you should be able to write without looking up?

Answer. With n a non-negative int: $n \& 1$ tests parity; $n \& (n-1)$ clears the lowest set bit (and is 0 exactly when n is a power of two); $n | (n-1)$ rounds up to the next all-ones mask; $n \wedge (n-1)$ isolates the run of low bits. For the count of set bits, either loop-and-clear or `bin(n).count("1")`.

```

assert bin(12).count("1") == 2
assert (16 & 15) == 0 and (12 & 11) == 8    # 10000->0, 1100->1000
assert (5 & (5 - 1)) == 4                    # clear lowest set bit: 101 -> 100
assert (7 | (7 - 1)) == 7 and (6 | 5) == 7  # round up to next all-ones
assert ((n := 8) & (n - 1)) == 0            # power of two -> 0

```

The power-of-two test generalises to "is this a valid bitmask / block size?" checks all over systems code. The XOR identity $a \wedge a == 0$ and $a \wedge 0 == a$ is also how you find the single unpaired element in one pass.

[Hard] Q. You need a/b as a fraction reduced to lowest terms, and a and b

are up to 10^{18} . What do you do, and what are the overflow traps?

Answer. Compute $g = \text{gcd}(a, b)$ and return $(a//g, b//g)$. The Euclidean algorithm is safe at that size in Python (arbitrary precision), but in fixed-width languages the trap is not the GCD - it is anywhere you **multiply** before dividing. If a problem needs $a * b / c$ and you do the multiplication first, you can overflow even when the final quotient fits. Reduce by gcd factors before multiplying, or use a bignum / `__int128` / `Math.multiplyHigh` equivalent.

```

from math import gcd

def reduce(a: int, b: int) -> tuple[int, int]:
    g = gcd(a, b)
    return a // g, b // g

assert reduce(462, 1071) == (22, 51)    # gcd is 21
assert reduce(20, 50) == (2, 5)
assert reduce(0, 7) == (0, 1)
assert reduce(10**18, 10**18 - 10**9) == (10**9, 10**9 - 1)

```

The follow-up they will ask: the same for adding fractions $a/b + c/d = (a*d + c*b) / (b*d)$ - the numerator and denominator can be quadratically larger than the inputs, so cross-reduce with $\text{gcd}(a, d)$ and $\text{gcd}(c, b)$ before the big multiply.

[Easy] Q. When is a number a power of two, and how do you find that power?

Answer. A positive integer is a power of two exactly when it has a single set bit: $n > 0$ and $(n \& (n - 1)) == 0$. To find the exponent use the bit length minus one ($n.bit_length() - 1$), which is $O(1)$ in Python.

```
def is_power_of_two(n: int) -> bool:
    return n > 0 and (n & (n - 1)) == 0

assert is_power_of_two(1) and is_power_of_two(1024)
assert not is_power_of_two(0) and not is_power_of_two(12)

assert (64).bit_length() - 1 == 6      # 64 == 2**6
```

Why it works: a power of two looks like $1000\dots 0$; subtracting 1 gives $0111\dots 1$, so the AND is 0. Any number with two or more set bits keeps at least one bit in common with $n-1$. The $n > 0$ guard matters - 0 is $1000 \& 0111$ -free but is not a power of two.

[Medium] Q. Why shouldn't I use floating point for money, and what do I use instead?

Answer. Because binary floating point cannot represent most decimal fractions: 0.1 is $0.1000000000000000055511151231257827\dots$ in a double, so $0.1 + 0.2 == 0.3$ is false. For money the error is not philosophical - it is a cent that does not match the ledger, and it compounds silently over aggregations (summing 10^5 invoices in float drifts by a visible amount).

The standard fix is **fixed point in integer units**: store the smallest unit (cents, millis, satoshis) as an integer. All arithmetic is exact integer arithmetic; you only divide by 100 when rendering, and round with a named rule (half-up is the usual choice for customer-facing display; be consistent, because "which rounding" is as much a decision as the number).

Why not just use a `Decimal` type? Often that is fine and simpler - `Decimal` is exactly fixed-point with a decimal exponent, and it is the right answer for anything with variable precision (tax rates, exchange rates). The integer-units approach wins when you need the value to be:

- **Stable on the wire and in the database** - `BIGINT cents` is unambiguous
- **Comparable and sortable without scale negotiation** - `Decimal("1.10")`
- **Safe against overflow reasoning** - with integers you can state the bound

The trap to name even if not asked: **rates are the exception**. "3.99%" is not a count of a smallest unit; multiplying `cents x rate` needs a real rational step (integer rate in basis points, or a `Decimal`), then a final round to the smallest currency unit - and the rounding rule must be decided before implementation, because half-up vs half-even changes totals over a month of transactions.

Worked example of the float failure: `examples/python/numbers/float_vs_decimal.py`.

Follow-up. "Three friends split \$100.00. How do the cents work out?" Expected: $33.34 + 33.33 + 33.33$ - the remainder of the integer division must be allocated to named parties by rule (e.g. first N parties get the extra cent), or the split does not sum back to the total, which is exactly the ledger-mismatch the integer approach is meant to prevent.

Recursion and Backtracking

Recursion is how you express "solve a smaller version of this". Backtracking is recursion that undoes its choices - the standard tool for permutations, combinations and constraint puzzles.

[Easy] Q. What are the parts of a correct recursive function?

Answer. Three, and missing any one is a bug:

- 1 **A base case** that returns without recursing.
- 2 **A recursive case** that calls itself on a **strictly smaller** input.
- 3 **Progress** - every path must move toward the base case.

```
def factorial(n):  
    if n <= 1:          # base case  
        return 1  
    return n * factorial(n - 1)  # smaller input, guaranteed progress
```

Infinite recursion is almost always a missing base case or a recursive call that does not shrink the input (`factorial(n)` instead of `factorial(n - 1)`).

[Easy] Q. What actually happens in memory when a function recurses?

Answer. Each call pushes a **stack frame** holding that call's parameters, local variables and return address. Frames are popped as calls return.

That is why recursion costs **O(depth) space** even when it allocates nothing, and why exceeding the stack limit raises `StackOverflowError` / `RecursionError`. Python's default limit is 1000 frames - deliberately low, to turn runaway recursion into a clean error instead of a crash.

The practical consequence: recursion depth proportional to n is fine for a balanced tree (depth $\log n$) and dangerous for a linked list or a degenerate tree (depth n).

[Medium] Q. What is tail recursion, and does it help in your language?

Answer. A call is in **tail position** when it is the very last thing the function does - its result is returned directly, with no pending work.

```
def sum_to(n, acc=0):  
    if n == 0:  
        return acc  
    return sum_to(n - 1, acc + n)  # tail call: nothing waits on it
```

A compiler can reuse the current frame instead of pushing a new one, turning the recursion into a loop: $O(1)$ space. Scheme and Lua guarantee this; Scala, Kotlin and most functional languages do it (`@tailrec`); JavaScript engines mostly do not, despite it being in the spec.

Python does not do it and will not - Guido van Rossum has rejected it deliberately, partly because it destroys the stack traces that make debugging possible. So in Python, rewriting for tail position buys you nothing. Convert to an explicit loop instead.

[Medium] Q. When should you convert recursion to iteration?

Answer. When depth can be large, or when the recursion is simple enough that a loop is clearer.

Every recursion can be made iterative with an explicit stack. That does not reduce total space - it moves it from the limited call stack to the heap, which is exactly the point when depth is the problem.

Prefer **recursion** for trees, nested structures and divide-and-conquer, where it mirrors the shape of the data. Prefer **iteration** for linear traversals, and whenever depth is $O(n)$ on untrusted input.

[Medium] Q. What is backtracking?

Answer. Systematic search: make a choice, recurse, then **undo the choice** before trying the next one. It explores a tree of partial solutions and abandons a branch as soon as it cannot lead anywhere ("pruning").

The template is the same every time:

```
def backtrack(state, choices, results):
    if is_complete(state):
        results.append(state.copy())    # copy! state keeps mutating
        return
    for choice in choices:
        if not is_valid(state, choice):
            continue                    # prune
        state.append(choice)            # 1. choose
        backtrack(state, choices, results) # 2. explore
        state.pop()                    # 3. un-choose
    return results
```

Two bugs account for most failures: **forgetting to undo** (state leaks between branches) and **appending** `state` **instead of** `state.copy()` (every result is the same list object, which ends up empty).

[Medium] Q. Generate all permutations of a list.

Answer.

```
def permutations(nums):
    results, used, current = [], [False] * len(nums), []

    def backtrack():
        if len(current) == len(nums):
            results.append(current.copy())
            return
        for i, n in enumerate(nums):
            if used[i]:
                continue
            used[i] = True
            current.append(n)
            backtrack()
            current.pop()    # undo
            used[i] = False  # undo
        return

    backtrack()
    return results
```

$O(n \times n!)$ time - there are $n!$ permutations and copying each costs $O(n)$ - and **$O(n)$ space** beyond the output.

You cannot beat $n!$ when the output is $n!$ items; that is an output-size bound, not an inefficiency. Say so explicitly, because candidates often apologise for a complexity that is provably optimal.

[Medium] Q. Subsets vs permutations vs combinations - how do the recursions differ?

Answer. All three are the same skeleton with a different loop bound.

	Count	Key difference
Subsets	2^n	Every element is take-or-skip; record at every node
Permutations	$n!$	Order matters; loop over all unused elements
Combinations (choose k)	$C(n,k)$	Order does not matter; loop from <code>start</code> onward

The single line that separates permutations from combinations is whether the recursive call restarts the loop at 0 (permutations) or at `i + 1` (combinations). Passing a `start` index is what stops `[1,2]` and `[2,1]` both appearing.

[Hard] Q. How does pruning change backtracking's performance?

Answer. It does not improve the worst case, but it is the difference between usable and useless in practice.

N-Queens is the standard illustration. Brute force places a queen in every square of every row: $8^8 \sim 16.7$ million placements for an 8x8 board. Checking validity as you place - abandoning a row the instant a queen is attacked - cuts it to roughly 15,000 nodes explored, and only 92 solutions exist.

The principle: **check constraints as early as possible**, not at the leaves. A branch you cut at depth 2 removes everything beneath it. This is also why the order you try choices matters - trying the most constrained option first ("minimum remaining values") prunes hardest and is the core heuristic in Sudoku solvers.

Selection Algorithms

[Medium] Q. How would you find the kth smallest value without fully sorting?

Answer. Use **quickselect** for expected $O(n)$ time and $O(1)$ auxiliary space when mutation is allowed, or keep a size-k heap for $O(n \log k)$ time when input arrives as a stream or must not be rearranged.

Quickselect partitions around a pivot like quicksort, but continues only into the side containing rank k. Balanced partitions cost $n + n/2 + n/4 + \dots = O(n)$. Repeated bad pivots give an $O(n^2)$ worst case, so real implementations randomize the pivot or use a defensive strategy.

The trade-off is the interview-worthy part: sort for many rank queries, use a heap for streaming top-k, and quickselect for a one-off query on a mutable in-memory array.

Sorting and Searching

You will rarely implement a sort in production. You will frequently be asked to compare them, and binary search is one of the most bug-prone things in computing.

[Easy] Q. Compare the sorting algorithms you know.

Answer.

Algorithm	Best	Average	Worst	Space	Stable?
Bubble / insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting / radix	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes

The three sentences worth having ready:

- **Merge sort** guarantees $O(n \log n)$ and is stable, but needs $O(n)$ scratch space.
- **Quicksort** is usually fastest in practice - great locality, in-place - but
- **Heap sort** is the only one that is both $O(n \log n)$ worst case and $O(1)$

[Medium] Q. If quicksort has an $O(n^2)$ worst case, why is it the default almost everywhere?

Answer. Because the worst case is avoidable and the average case is excellent.

$O(n^2)$ happens when the pivot is consistently the smallest or largest element - classically, choosing `arr[0]` on already-sorted input, which is depressingly common in real data. The fixes:

- **Randomised pivot** - an adversary cannot predict it.
- **Median-of-three** - sample first, middle, last.
- **Introsort** - start with quicksort, count recursion depth, and switch to heap

Quicksort also wins on constants: it partitions in place with sequential memory access, so it is very cache-friendly, whereas merge sort writes to a second array.

The real-world answer: most standard libraries no longer use plain quicksort. Python and Java (for objects) use **Timsort**, a stable merge sort that detects already-sorted runs and is $O(n)$ on nearly-sorted data. Java uses dual-pivot quicksort for primitives, where stability is meaningless.

[Medium] Q. What does "stable" mean and when does it matter?

Answer. A stable sort preserves the relative order of elements that compare equal.

It matters whenever you sort **more than once**. To sort employees by department, and by salary within each department, a stable sort lets you do it in two passes:

```
people.sort(key=lambda p: p.salary)      # secondary key first
people.sort(key=lambda p: p.department)  # primary key second
```

The second sort preserves the salary ordering inside each department - only because it is stable. With an unstable sort this silently produces wrong output.

Stability is also why Python's `sorted` and Java's `Collections.sort` guarantee it in their specification: users depend on this pattern.

[Medium] Q. Can you sort faster than $O(n \log n)$?

Answer. Not with **comparisons**. There are $n!$ possible orderings, and each comparison yields one bit, so you need at least $\log_2(n!) \sim n \log n$ comparisons. That is a proven lower bound, not a limitation of current algorithms.

You can beat it by not comparing. **Counting sort** tallies occurrences of each value into buckets: $O(n + k)$ for values in a range of size k . **Radix sort** applies counting sort digit by digit: $O(d \times (n + k))$.

The catch is the constraints. Counting sort on 32-bit integers needs a 4-billion-entry array. These are worth it only when the key range is small and known - sorting ages, exam scores, or fixed-length strings.

If an interviewer asks for better than $O(n \log n)$, they are asking you to notice a constraint on the values.

[Easy] Q. Write binary search. What are the classic bugs?

Answer.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1          # inclusive bounds
    while low <= high:                  # <=, not <
        mid = low + (high - low) // 2    # overflow-safe
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1                 # must be +1
        else:
            high = mid - 1                # must be -1
    return -1
```

$O(\log n)$ time, $O(1)$ space. Four bugs live in those eight lines:

- 1 `while low < high` instead of `<=` - misses a one-element range, so the
- 2 `(low + high) // 2` - overflows in fixed-width languages (C, Java) when
- 3 `low = mid` instead of `mid + 1` - infinite loop, because the range stops
- 4 **Forgetting the array must be sorted.** Binary search on unsorted data

Test with: empty array, one element, target absent, target at index 0, target at the last index.

[Medium] Q. How do you find the first occurrence of a value in an array with duplicates?

Answer. Do not stop at the first match - record it and keep searching left.

```
def first_occurrence(arr, target):
    low, high = 0, len(arr) - 1
    found = -1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            found = mid                    # candidate
            high = mid - 1
```



```

        high = mid - 1                # but look further left
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
    return found

```

Still $O(\log n)$. Mirror it (`low = mid + 1` on a match) for the last occurrence. This "keep searching after you find it" shape is the basis of `bisect_left / lower_bound`, and it generalises to the more important question below.

[Hard] Q. What is "binary search on the answer"?

Answer. Binary search does not require an array. It requires a **monotonic predicate**: a boolean function that is false, false, ..., false, true, true, ..., true over the search range. Then you can binary search for the boundary.

This unlocks a whole class of optimisation problems - "minimise the maximum X", "what is the smallest capacity that works?":

```

def min_feasible(low, high, works):
    """Smallest value in [low, high] for which works(value) is True."""
    best = -1
    while low <= high:
        mid = low + (high - low) // 2
        if works(mid):
            best = mid
            high = mid - 1          # try to do better
        else:
            low = mid + 1          # not enough
    return best

```

"Ship packages within D days", "minimum eating speed", "split array to minimise the largest sum" are all this function with a different `works`.

The tell: the problem asks for a minimum or maximum value, the answer lies in a numeric range, and checking "is value V feasible?" is easy even though finding the best V directly is not. Complexity becomes $O(\log(\text{range}) \times \text{cost of works})$.

A worked binary search with insertion points is in

`examples/python/algorithms/binary_search_iterative.py`.